

WorkTrack ERP - Developer Handoff Blueprint

Detailed SRS + module workflow + data model + API blueprint based on uploaded Apps Script, WorkTrack UI, Client Portal, and Analytics Dashboard code

Document item	Value
Prepared for	ERP developer / implementation team
Prepared on	02 Jul 2026
Source reviewed	6 uploaded files: backend, WorkTrack UI, Client Portal backend, Client Portal UI, Dashboard backend, Dashboard UI
Approximate source size	24,536 lines
Detected functions / handlers	391
Goal	Build a secure, scalable ERP without missing any current workflow

1. Executive Summary

Current system is a Google Apps Script + Google Sheets based ERP MVP with three major portals: WorkTrack Employee/Admin Portal, Client Portal, and Master Analytics Dashboard. It already includes users, hierarchy, attendance, leave/intimation, tickets, expenses, FMS, To-Do, client tickets, payment restriction, invoices, reports, WhatsApp notifications, and trigger-based reminders.

Developer objective: convert this MVP into a stable ERP product with central authentication, role based access control, normalized database, audit logs, clean APIs, reusable UI modules, and reliable reporting. The sections below describe every current module, required screens, backend services, database tables, workflows, validation rules, and missing improvements.

2. Source Code Inventory

File	Lines	Functions	UI IDs	Purpose
Pasted text.txt	6089	147	0	Main WorkTrack backend: users, approvals, dashboard, attendance, tickets, expenses, FMS, To-Do, notifications, utilities
Pasted text (2).txt	9709	99	356	WorkTrack frontend HTML/JS/CSS: login, sidebar, dashboards, attendance, tickets, employee master, approvals, reports, FMS, forms, clients
Pasted code (3).js	823	30	0	Client Portal backend: client auth, ticket creation, invoices, reports, checklist/social tasks, messages, notifications
Pasted text (4).txt	3553	53	101	Client Portal frontend: login, dashboard, ticket views, checklists, social tasks, invoices, reports, chat
Pasted code (5).js	514	19	0	Master Analytics backend: reports aggregation, date parsing, ticket/todo creation, unified task mapping
Pasted text (6).txt	3848	43	104	Master Analytics frontend: filters, KPIs,

				charts, workload tables, explorer, batch tasks, CSV export
--	--	--	--	--

3. Target ERP Architecture

Layer	Recommended implementation	Reason
Frontend	React / Next.js web app with role-based routes	Modern UI, reusable components, faster development than large single HTML files
Backend API	Node.js/NestJS or Express API services	Clean business logic, validation, background jobs, audit logging
Database	PostgreSQL / Supabase	Relational workflows, reporting, constraints, joins, backups
Authentication	Hashed passwords + refresh tokens + optional OTP	Avoid plain sheet passwords and default 123456 risk
File storage	Google Drive API / S3 compatible storage	Attachments, attendance photos, receipts, proofs
Notifications	WhatsApp provider + email queue + retry logs	Reliable alerts without blocking user actions
Reporting	Materialized report views + dashboard APIs	Avoid reading every sheet for every dashboard request
Admin tooling	Settings UI for roles, permissions, templates, holidays, TAT rules	No hardcoded IDs or business rules in source code

4. Global ERP Roles and Permission Matrix

All modules must use one central RBAC engine. Do not hardcode employee IDs for restrictions. Store permission rows in database and evaluate them server side for every API call.

Role	Access scope	Key permissions
Super Admin	Entire company	Manage all data, reports, settings, users, clients, final approvals except rules explicitly restricted by permission table
Admin	Assigned departments/teams	Manage tickets, team attendance, reports, task assignment, client work
HR	Employees except restricted categories if configured	Employee master, attendance approvals, leave/intimation, onboarding, HR reports
Manager / Team Lead	Own team	Approve team tickets/leaves/attendance, reassign tasks, see team reports
Employee / User	Self	Punch attendance, submit leave/intimation/expense, work on assigned tickets/FMS/todo, chat
Client	Own client account only	Create tickets, view own tasks, approve/reject, respond, invoices, reports, chat
Finance / Accounts	Finance scope	Invoices, payments, expenses, client payment restrictions, revenue reports

5. Global Status Standardization

Developer should create a controlled status master. Current code checks many free-text statuses. ERP should map them into consistent groups.

Entity	Allowed statuses	Notes
Ticket	Open, Assigned, In Progress, Paused, Pending Client Response, Client Responded, Pending Approval, HR Approved, Rework, Closed, Cancelled	Use status_history table for every transition
Attendance	Present, Late, Need Approval, Pending, Approved, Rejected, Missed Punch, Half Day	Separate raw punch action from approval status
Leave	Pending, Approved, Rejected, Cancelled	Balance should update only after approval

Intimation	Submitted, Approved, Rejected, Cancelled	Late coming / WFH / short leave types
Expense	Pending, Verified, Approved, Rejected, Paid	Finance payment separate from manager approval
FMS	Future, Pending, In Progress, Completed, Overdue	Actual Date determines completion in current code
Todo	Pending, Completed, Deleted	Deleted should be soft delete
Invoice	Draft, Sent, Partial, Paid, Full Payment Received, Overdue, Cancelled	Payment restriction should read canonical status

6. Module 1 - Authentication and Session Management

Current behavior: employee/client login data is read from sheets. New app users are auto-created from employee master and currently receive default password. Developer must replace this with secure auth.

Feature	Current need	Developer implementation
Employee login	Employee ID + password	/auth/login with hashed password, JWT/session token, device metadata
Client login	Client ID + password	Separate client_auth table or user_type=client with tenant_id
Password storage	Plain/default passwords exist in current MVP	bcrypt/argon2 hash; never store or send plain passwords
First login	New user must change default password	force_password_reset flag + one-time setup link
Forgot password	Not reliable in sheet MVP	OTP/email reset token with expiry
Session expiry	Browser state currently likely local	Access token short expiry + refresh token + logout invalidation
Access enforcement	Mostly UI and function checks	Every API must call authorize(user, action, resource) server-side

7. Module 2 - Employee Master and User Management

This is the root module for WorkTrack hierarchy. Employee Master should sync with Users table but not overwrite manually controlled fields unless allowed.

Screen	Fields / actions	Developer notes
Employee List	Search, filter by category/status/department, view active/inactive	Tabs: Master, EMP, Intern, Freelancer, Inactive, Users
Add/Edit Employee	Name, User ID, Emp Code, phone, email, department, designation, joining date, status, manager, task approver, documents	Validate unique User ID and Employee ID
User Account Sync	Create login account from Employee Master	Set status active/inactive based on employee status
Role Assignment	Role, manager ID, task approver, department, permissions	Role changes must be audit logged
Inactive/Exit Flow	Resigned, terminated, inactive	Disable login, keep historical work and reports
Documents	Upload Aadhaar/PAN/offer/other docs if needed	Private storage, not public links by default

1. HR/Admin opens Employee Master.
2. Adds employee with required personal and official details.
3. System validates duplicate Employee ID/User ID/mobile/email.
4. System creates user account with role=User by default unless admin selects another role.
5. System sends secure onboarding link, not plain password.
6. Manager and Task Approver are assigned.
7. User appears in active users for tickets/FMS/to-do assignment.
8. When status becomes resigned/inactive/terminated, login is disabled but historical records remain visible in reports.

8. Module 3 - Hierarchy and Approval Engine

Approval should be implemented as one reusable service, not separately inside each module. Current logic uses Task Approver first, Manager fallback, and reassigned ticket approver overrides previous manager. Keep this behavior.

Approval type	Approver logic	Required actions
Attendance	Manager/Admin for team, HR/Super Admin based on permission	Approve, reject, add remarks, edit status if permitted
Leave	Manager/Admin team, HR/Super Admin	Approve/reject, update leave balance, notify employee
Intimation	Manager/Admin team, HR/Super Admin	Approve/reject, link to attendance if applicable
Ticket completion	Task Approver first, Manager fallback,	Approve close, reject/rework, transfer

	Reassigned To overrides old approver	approval
Expense	Manager/HR/Finance/Super Admin based on amount and policy	Verify, approve, reject, mark paid
Client approval	Client on own tickets/social/checklists	Approve, reject, request changes, respond

9. Module 4 - Attendance Management

Component	Developer specification
Punch In/Out	Capture action, date, time, employee, photo URL, latitude, longitude, device/IP, remarks, approval status
Attendance Gate	If employee has not punched in, block starting task if company policy requires it
Geo/photo proof	Store proof privately; provide signed/permissioned URL to approver
Late/missed punch	Auto-detect late, missing punch, missing punch out; generate approval request
Daily status	Present, Absent, Late, Half Day, On Leave, WFH, Holiday
Approval dashboard	Group by employee + date, show punch-in/out, duration, proof, approve/reject
Reports	Daily attendance, monthly attendance, employee attendance history, late count, missing punch count

9. Employee opens Attendance page.
10. System checks today attendance state.
11. Employee captures photo or allows location, then punches in.
12. System saves raw punch record and calculates status: on time, late, need approval, etc.
13. If punch out, system calculates working duration.
14. If approval needed, record goes to approver queue.
15. Approver reviews proof and approves/rejects with remarks.
16. Dashboard and payroll/HR reports consume approved attendance summary.

10. Module 5 - Leave and Intimation

Submodule	Fields	Workflow
Leave Request	Employee, leave type, start date, end date, reason, attachment, status, approver, remarks	Submit -> Manager/HR approval -> balance update -> notify employee
Intimation	Type, date, reason, in/out time if applicable, status, approver, remarks	Submit -> approval -> attendance adjustment if applicable
Leave Balance	Opening, earned, used, pending, remaining	Should be separate from requests table
Calendar	Approved leaves, holidays, WFH	Used in attendance and workload planning

11. Module 6 - Ticket / Task Management

Ticket module is the core operating system for client and internal work. It should support creation by employee, manager, admin, client portal, and automated batch import.

Feature	Required behavior
Create Ticket	Fields: ticket ID, client, creator, assignee, category, priority, description, plan date, TAT, attachments, source
Bulk Create	Allow multiple rows from UI/client portal; validate required fields per row before save
Assignment	Assign to employee; notify assignee; visible based on role/team/client
Timer	Start, pause, resume, end. Store sessions in ticket_time_logs, not only total duration text
Reassign to User	Track reassigned_by, reassigned_to, old assignee, reason, timestamp
Send to Client	Move to Pending Client Response; client can respond/approve/reject

Client Responded	Employee/manager sees response and continues work
Submit for Approval	Employee ends work -> Pending Approval to Task Approver/Manager
Approve Close	Approver closes ticket or sends to Rework with remarks
History	Every change goes into ticket_history with old/new value and actor
Chat	Task-level messages with read/unread state for employee, client, manager
TAT reminders	Notify before/after planned start and overdue; escalate if not started/completed

17. Ticket is created by employee/admin/client/batch.
18. System generates stable unique ticket ID; row number must not be primary ID.
19. Ticket is assigned to employee or queue.
20. Assignee receives notification with TAT, priority, client, and description.
21. Assignee starts timer only if attendance gate passes.
22. Assignee may pause/resume; system stores each session.
23. If input from client is required, assignee sends to client with clear question/attachment.
24. Client responds from Client Portal; status becomes Client Responded.
25. Assignee completes work and submits for approval.
26. Approver reviews output, duration, remarks, and either closes or sends rework.
27. Closed ticket appears in client report and ERP dashboard.

12. Module 7 - Client Portal

Client Portal is a tenant-limited portal. Client must only see records for their own Client_Id.

Screen	Purpose	Developer requirements
Client Login	Client authentication	Secure password hash, active status check, no data leakage
Client Dashboard	Summary of open/completed/pending/invoice states	Cards: Open tickets, completed tickets, awaiting response, invoices due, recent activity
Create Tickets	Bulk ticket creation with attachments	Payment restriction check before ticket actions
Tickets	Open/completed filters, status badge, detail, chat, response	Client can respond, approve/reject where allowed
Checklist Tasks	Service checklist visibility	Client can view details/status and approvals if configured
Social Media Tasks	Post approval workflow	Approve/reject social posts, view history
Invoices	Invoice list and status	Due amount, date, status, downloadable invoice, payment link if added
Reports	Client monthly report	Charts + table + export PDF/CSV
Notifications	Unread updates	Polling/websocket; mark as notified/read

13. Module 8 - Payment Restriction and Finance

Current client backend blocks ticket actions if an unpaid invoice is older than 10 days. ERP should keep this as configurable policy, not hardcoded.

Functionality	Developer specification
Invoice master	Invoice ID, client, date, due date, amount, tax, paid amount, balance, status, PDF URL
Payment log	Payment ID, invoice ID, mode, transaction ref, amount, date, recorded by
Restriction policy	Policy table: client, grace days, restrict ticket create/respond/approve yes/no
Restriction check	Before client ticket creation/action, calculate overdue unpaid invoices
Alerts	Send reminders before due, on due, after due, and before restriction
Finance dashboard	Receivables, overdue clients, monthly revenue, payment aging

14. Module 9 - Expense Management

Screen / action	Fields and behavior
Submit Expense	Date, type, amount, description, receipt, employee, status pending
Review Expense	Approver sees receipt, amount, employee, remarks
Approval Flow	Pending -> Verified -> Approved/Rejected -> Paid
Notifications	Notify HR/Finance/Super Admin based on policy
Reports	Employee expense history, pending claims, monthly total, type-wise expense

15. Module 10 - FMS Workflow

FMS appears as structured recurring/process tasks. Current logic maps FMS fields flexibly and marks completion by Actual Date. Developer should formalize this into process templates and generated tasks.

Feature	Developer specification
FMS Template	Process name, step name, assigned role/person, planned duration, dependency, frequency
FMS Task	Generated task instance with planned date, actual date, assignee, status, TAT
Visibility	Self tasks, team tasks, admin all tasks based on role and assignment token
Future Task Logic	Tasks with future planned date should not count as overdue
Completion	Mark done with actual date/time, remarks, attachment if required
Special TAT Rules	Configurable overrides e.g., CRM follow-up/call = 5 minutes
Reports	Pending, overdue, completed, employee-wise and client/process-wise workload

16. Module 11 - To-Do System

Feature	Developer specification
Create Todo	Title/description, assignee, due date, priority, created by, source
Bulk Todo	Admin can add multiple todos
My Todos	Employee sees own pending/completed
Team Todos	Manager/Admin sees team
Super Admin Audit	Company pending To-Dos summary
Soft Delete	Mark deleted, do not remove row permanently
Reports	Pending by assignee, overdue, completed count

17. Module 12 - Chat, Comments, and Messages

Task-level communication should be unified across WorkTrack and Client Portal.

Requirement	Specification
Message entity	message_id, task_type, task_id, sender_type, sender_id, body, attachments, created_at
Read state	message_read table per user/client; unread badge count
Task types	Ticket, checklist, social task, FMS if enabled
Visibility	Only participants with task access can read/post
Notifications	New message triggers WhatsApp/email/in-app as configured
Audit	Edited/deleted messages tracked; avoid permanent delete

18. Module 13 - Forms and Client Portal Configuration

Current WorkTrack includes Forms and Clients Portal admin functions. Developer should make these configurable screens.

Admin area	Purpose
Forms Portal Data	Manage form entries, status, filters, permissions
Client Portal Data	Manage client portal records and access

Dynamic Filters	Department, service, client, status, date
Permissions	Only allowed admins can add/edit/delete
Audit	Every admin change captured

19. Module 14 - Master Analytics Dashboard

This dashboard is the executive ERP report layer. It should not directly scan raw operational tables every time in production. Create reporting views and materialized summaries.

Dashboard area	Metrics and behavior
Overview	Active users, active clients, open tasks, completed tasks, overdue, pending approvals, total planned/actual hours
Workload Table	Combined Tickets + FMS + To-Do with normalized fields: id, desc, type, client, employee, status, planned date, TAT, duration
Employee Explorer	Employee profile, attendance, workload, performance, overdue breakdown
Client Explorer	Client-wise tickets, tasks, effort, status, invoices
Performance Analytics	Efficiency, planned vs actual, delay, instant closures, poor/good performance flags
Filters	Date range, employee, client, status, task type, department
Batch Creation	Create ticket/todo rows from dashboard if permitted
Exports	CSV/PDF export by table and report

20. Module 15 - Notifications, Triggers, and Automation

Automation	Trigger / condition	Action
Daily Pending Reminder	Morning/evening scheduled job	Send user attendance + pending ticket/todo summary
Super Admin Audit	Scheduled job	Send company pending To-Do report
Company Summary	Scheduled job	Send attendance + task summary to leadership
Ticket Assignment	Ticket assigned/reassigned	Notify assignee with task details
TAT Reminder	Not started / due soon / overdue	Notify employee, then manager, then super admin
Expense Claim	New expense submitted	Notify HR/Finance/Super Admin
Payment Reminder	Invoice due/overdue	Notify client/account manager
Client Update	Client responds or approves/rejects	Notify task owner/manager
Retry Failed WhatsApp	Scheduled retry	Use notification_logs with status failed/retried/sent

21. Database Schema Blueprint

Below is the minimum normalized schema. Developer can implement with PostgreSQL/Supabase. Use UUID primary keys internally and readable business codes for display.

Table	Fields
users	id, employee_code, user_id, name, email, mobile, password_hash, role_id, department_id, manager_id, task_approver_id, status, force_password_reset, created_at, updated_at
roles	id, name, description, is_system
permissions	id, role_id/user_id, module, action, scope, allow
departments	id, name, status
clients	id, client_code, name, services, account_manager_id, status, billing_cycle, portal_enabled
client_contacts	id, client_id, name, email, mobile, role, login_enabled
tickets	id, ticket_code, source, client_id, creator_user_id, creator_client_id, assignee_id, category, priority, description, plan_date, tat_minutes, status, current_approver_id, close_date, last_update_at
ticket_history	id, ticket_id, actor_type, actor_id, action, old_value, new_value, remarks, created_at
ticket_time_logs	id, ticket_id, user_id, start_at, pause_at, end_at, duration_minutes, remarks
messages	id, task_type, task_id, sender_type, sender_id, body, attachment_url, created_at
message_reads	id, message_id, reader_type, reader_id, read_at
attendance	id, user_id, date, action, time, photo_url, latitude, longitude, duration_minutes, status, approval_status, approver_id, remarks
attendance_summary	id, user_id, date, punch_in, punch_out, work_minutes, status, approval_status
leave_requests	id, user_id, leave_type, start_date, end_date, reason, status, approver_id, remarks, created_at
leave_balances	id, user_id, leave_type, opening, earned, used, pending, remaining
intimations	id, user_id, type, date, from_time, to_time, reason, status, approver_id, remarks
expenses	id, expense_code, user_id, date, type, amount, description, receipt_url, status, approver_id, payment_status
invoices	id, invoice_code, client_id, invoice_date, due_date, amount, tax, paid_amount, status, pdf_url
payments	id, invoice_id, amount, mode, transaction_ref, paid_at, recorded_by
fms_templates	id, name, client_id, department_id, status
fms_tasks	id, template_id, task_code, description, assignee_id, planned_date, actual_date, tat_minutes, status, remarks
todos	id, todo_code, description, assignee_id, due_date, priority, status, created_by, source
attachments	id, owner_type, owner_id, file_name, mime_type, storage_url, uploaded_by, created_at
notification_logs	id, channel, recipient, template, payload, status, error, retries, sent_at, created_at
audit_logs	id, actor_type, actor_id, module, action, resource_type, resource_id, before_json, after_json, ip, created_at
settings	id, key, value_json, description, updated_by, updated_at

22. API / Backend Service Blueprint

Developer should create APIs grouped by bounded context. Existing Apps Script functions can be used as business logic reference, but final ERP should use REST/GraphQL APIs with validation and tests.

Service	Endpoints
Auth	POST /auth/login, POST /auth/logout, POST /auth/refresh, POST /auth/change-password, POST /auth/reset-request, POST /auth/reset-confirm
Users	GET /users, POST /users, PATCH /users/{id}, GET /users/{id}, POST /users/{id}/disable, GET /users/team
Permissions	GET /roles, POST /roles, GET /permissions, PATCH /permissions
Attendance	POST /attendance/punch, GET /attendance/me, GET /attendance/team, POST /attendance/{id}/approve, POST /attendance/{id}/reject

Leave	POST /leaves, GET /leaves/me, GET /leaves/pending, POST /leaves/{id}/approve, POST /leaves/{id}/reject
Intimation	POST /intimations, GET /intimations/me, GET /intimations/pending, POST /intimations/{id}/approve
Tickets	GET /tickets, POST /tickets, POST /tickets/bulk, GET /tickets/{id}, PATCH /tickets/{id}, POST /tickets/{id}/start, POST /tickets/{id}/pause, POST /tickets/{id}/end, POST /tickets/{id}/submit-approval, POST /tickets/{id}/approve, POST /tickets/{id}/rework, POST /tickets/{id}/reassign, POST /tickets/{id}/send-to-client
Client Portal	POST /client/auth/login, GET /client/dashboard, GET /client/tickets, POST /client/tickets/bulk, POST /client/tickets/{id}/respond, POST /client/tickets/{id}/approve, GET /client/invoices, GET /client/reports
Messages	GET /messages?task_type=&task_id=, POST /messages, POST /messages/read
Expenses	POST /expenses, GET /expenses/me, GET /expenses/pending, POST /expenses/{id}/approve, POST /expenses/{id}/reject, POST /expenses/{id}/mark-paid
FMS	GET /fms/tasks, POST /fms/tasks, POST /fms/tasks/{id}/complete, GET /fms/templates
ToDo	GET /todos, POST /todos, POST /todos/bulk, PATCH /todos/{id}, POST /todos/{id}/complete, DELETE /todos/{id}
Reports	GET /reports/dashboard, GET /reports/admin, GET /reports/employee/{id}, GET /reports/client/{id}, GET /reports/export
Notifications	GET /notifications, POST /notifications/read, POST /jobs/retry-notifications

23. Frontend Screen Blueprint

Area	Screens/components
Global	Login, reset password, change password, profile, notification bell, sidebar, role based menu
Employee Dashboard	KPI cards, upcoming tasks, attendance state, pending approvals if manager, charts
Attendance	Punch camera/location, history, request status
Tickets	My tickets, team tickets, approval tickets, create/bulk, filters, timer controls, details modal, chat modal
Approvals	Tabs: Attendance, Leaves, Intimations, Tickets, Expenses; actionable first
Employee Master	Employee tabs, CRUD, sync login, inactive view
Expenses	Submit claim, receipt upload, claim history, approval list
FMS	My FMS, Team FMS, Future/Pending/Overdue filters, complete action
To-Do	My to-do, add, edit, bulk, complete, delete
Reports	Ticket report, FMS report, attendance report, expense report, KPI detail modal, exports
Forms/Clients Admin	Manage portal data, filters, permission-controlled add/edit/delete
Client Portal	Login, dashboard, create tickets, open/completed tickets, checklist, social posts, invoices, reports, chat
Master Analytics	Overview, workload table, status filters, employee explorer, client explorer, batch modal, CSV export

24. Validation Rules

Module	Validation
Users	Unique employee_code/user_id; required name/status/role; valid email/mobile; manager cannot be self
Attendance	One active punch-in per user per day; punch-out after punch-in; location/photo required if policy enabled
Leave	End date >= start date; balance check if paid leave; overlapping leave warning
Ticket	Description, priority, category, plan date, assignee/client required as per source; TAT numeric; attachments size/type limit
Timer	Only assignee/authorized user can start; cannot start multiple active

	sessions on same ticket
Approval	Actor must be current approver or have explicit permission; cannot approve own ticket unless policy allows
Client	Client can only access own client_id records; payment restriction before ticket creation/action
Expense	Amount numeric > 0; receipt required above configured amount; duplicate claim warning
FMS	Planned date valid; completion requires actual date; dependency validation if configured
Invoice	Amount >= paid amount; due date >= invoice date; paid status only if balance zero

25. Security and Compliance Requirements

- Never store or send plain passwords. Replace default password flow with one-time password setup link.
- All access checks must happen server-side, not only by hiding UI buttons.
- Every create/update/delete/approval/login should write audit_logs.
- Files should not be public by default. Use signed URLs or permissioned Drive links.
- Client portal must enforce tenant isolation by client_id on every query.
- Use input validation and sanitization for messages, remarks, descriptions, uploaded file names, and filters.
- Use rate limits for login and OTP actions.
- Use backups and database migration scripts. Do not edit production schema manually.
- Use environment variables/secrets for spreadsheet IDs, WhatsApp tokens, email settings, and storage credentials.
- Create admin settings for hardcoded values like grace days, restricted roles, CRM TAT overrides, reminder timings.

26. Reporting Logic and KPI Definitions

KPI	Definition
Total Planned Bandwidth	Sum of TAT minutes for tasks planned in selected date range
Actual Productive Time	Sum of completed/in-progress task time logs in selected date range
Gap Time	Assumed working minutes minus actual productive minutes; should exclude approved leave/holiday if configured
Pending Tickets	Tickets in Open/In Progress/Rework/Client Responded/Pending Client Response
Completed Tickets	Tickets in Closed/Completed/Approved by Client/Done/Resolved group
Overdue Tasks	Pending tasks whose plan date is before today and not Future/Closed/Completed
Attendance %	Present/approved working days divided by expected working days
Efficiency %	Completed TAT or productive output divided by assumed bandwidth; define exact formula in settings
Client Open Work	Open tickets + pending checklist/social tasks for a client
Payment Due	Invoice amount minus paid amount where status not paid/cancelled

27. Migration Plan from Google Sheets to ERP

28. Freeze current sheet structure and export all tabs as CSV backup.
29. Create mapping document: sheet column -> database table field.
30. Build database schema and run migrations in development.
31. Import Users, Clients, Tickets, Attendance, Leaves, Intimations, Expenses, FMS, To-Do, Invoices, Messages, History.
32. Generate stable IDs for rows missing IDs; preserve old row number as legacy_reference.
33. Normalize statuses using status mapping table.
34. Validate counts: row counts, active users, open tickets, invoices, attendance totals.
35. Run dual-read pilot: old Apps Script and new ERP reports for same date range should match.
36. Switch writes to new ERP while keeping Sheets as read-only archive/export.
37. After 2-4 weeks, disable Apps Script write operations and keep only backup export if needed.

28. Implementation Roadmap

Phase	Duration	Deliverables
Phase 0 - Discovery	2-4 days	Confirm roles, sheet columns, status mapping, approval rules, hosting, WhatsApp provider, database choice
Phase 1 - Core Setup	1 week	Database, auth, RBAC, users, clients, audit logs, file storage
Phase 2 - WorkTrack Core	2-3 weeks	Attendance, leave/intimation, ticket lifecycle, timer, approvals, chat
Phase 3 - Client Portal	1-2 weeks	Client login, ticket creation, responses, invoices, reports
Phase 4 - FMS/To-Do/Expenses	1-2 weeks	FMS tasks, to-do, expenses, finance approvals
Phase 5 - Analytics	1-2 weeks	Master dashboard, filters, KPIs, exports, explorer views
Phase 6 - Notifications	3-5 days	WhatsApp/email queue, scheduled jobs, retry logs
Phase 7 - QA and Migration	1-2 weeks	Data import, UAT, role testing, performance testing, production deployment

29. Developer Acceptance Checklist

- Every current function/workflow has a mapped API or backend service.
- Every current screen has a replacement screen or documented reason for removal.
- No plain passwords, no default password in notification, no public file links unless explicitly intended.
- Role based routes and server-side permissions work for Super Admin, Admin, HR, Manager, User, Client.
- Client users cannot access other clients through URL/API manipulation.
- Ticket lifecycle works end to end: create -> assign -> start -> pause -> client response -> submit -> approve/rework -> close.
- Attendance works end to end with photo/location proof and approval.
- Reports match old sheet data for sample date ranges after migration.
- Audit logs exist for all critical actions.
- WhatsApp/email notification failure does not break the main transaction; failures are retried.
- Dashboard loads fast with indexes/materialized views, not full-table scans for every request.
- Backup and restore process tested before production launch.

30. Appendix A - Current Function Inventory by File

Pasted text.txt

Lines: 6,089; Detected functions/handlers: 147; UI IDs: 0

getEmpMasterData, saveFileToFolder, syncToAppUsers, getSheetDataFromSheetObject, getPendingApprovals, getDashboardData, getSafeDate, escapeRegExp

matchesUser, getPlanDate, getCompletionDate, adminTicketAction, _safeText_, _normText_, _appendWorkTrackLinkToMessage_, _getTicketAssignmentOwner_

isClientCreatedTicket, notifyTicketAssignmentOwner_, sendWhatsAppMessage, saveOrUpdateUser, syncUserStatusToEmpMaster, setupInitialSheets, getSpreadsheet, getSheet

getCachedSheetData, getSheetData, appendRowToSheet, updateRowInSheet, durationToMinutes, sendEmailNotification, doGet, getTaskMessages

postTaskMessage, markTicketMessagesAsRead, reassignTicket, processClientResponse, submitLeaveRequest, submitIntimation, processAdminAction, createNewAttendanceRow

getUserRequestStatus, recordAttendance, recordExpense, getExpensesForUser, getAttendanceForUser, createTicketInSheet, getTicketSystemData, getTicketDetails

updateTicketInSheet, exportReportForWeb, getKpiDetails, getSafeDate, getTicketReportData, getFmsReportData, escapeRegExp, matchesUser

checkForNewNotifications, fixMissingLeaveIds, fixMissingIntimationIds, _todoSafe_, _todoNorm_, _isActiveUser_, _isPendingTodo_, _isSameDateIST_

formatTodoDate, _getUserNameById_, _getAkashTodoAuditor_, _buildSuperAdminTodoAuditMessage_, notifyAkashForNewSuperAdminTodo_, parseStrictDateOnly, safeText, parseFMSDate

isDateToday, isDateOverdue, sendDailyPendingReminders, sendSuperAdminAudit, sendCompanySummary, _deleteTriggersByHandler_, setupReliableTriggers, checkAndSendReminders

cleanupOldProperties, logTicketHistory, createBulkTicketsInSheet, calculateSessionAndTotal, updateTicketSchedule, saveBase64FileToDrive, getTeamIds, getNextEmpCode

authenticateUser, getMyTeamMembers, getAllManagersList, _fmsSafe_, _fmsNorm_, _fmsSplitIds_, _fmsEscapeRegExp_, _fmsHasWholeToken_

buildFmsVisibilityContext, _fmsMatchesUser_, _getFmsRowDecision_, getFmsTasksForApp, markFmsTaskDoneInApp, saveEmpMasterData, saveEmpMasterDataWithFiles, fixDrivePermissions

FORCE_CLEAR_CACHE, autoClosePendingTickets, setupTodoSheet, getTodos, addTodo, toggleTodoStatus, editTodoItem, deleteTodoItem

addBulkTodos, fixBlankClientTickets, setupWaLogsSheet, logWhatsApp, retryFailedWhatsAppMessages, forceApproveAllPendingTickets, getAllUsersForAdmin, sendTicketStartReminder

checkTicketTatReminders, parseDateTimeSafe, processExpenseApproval, changeUserPassword, transferTicketApproval, getTaskApproversList, getFormsData, saveClientPortalData

saveFormsPortalData, deleteClientPortalData, deleteFormsPortalData, getClientsPortalData, addFormsPortalData, checkUserAttendanceActive, enforceAttendanceGate, hasValidId

getValidName, isFmsFutureDate, getWorkingDays, reformatDate, mapTaskData, safeParseDate, safeParseDate, parseDateSafe

isFmsFutureDate, getCompletionDate, shouldRun

Pasted text (2).txt

Lines: 9,709; Detected functions/handlers: 99; UI IDs: 356

showAppsScriptRuntimeError, updateLocalRow, updateSlide, showStyledModal, showIframeModal, showLoading, showSwalMessage, playNotificationSound

getUserName, getFormattedDate, handlePunch, handleLogin, setupUI, setView, handleLogout, loadDashboard

updateDashboardUI, handleKpiCardClick, renderUpcomingTasksTable, startPunchInTimer, stopPunchInTimer, checkAttendanceStatus, resetPunchPhotoUI, stopCamera

startCamera, capturePhoto, retakePhoto, openSecureCameraPopup, initCamera, openChangePasswordModal, closeChangePasswordModal, loadAttendanceHistory

handleLeaveRequestSubmit, handleIntimationSubmit, loadUserRequestHistory, loadTicketSystemData, switchTicketTab, renderTicketTable, populateTicketDropdowns, addTicketRow

updateRowUI, toggleSidebar, startTicketSubmission, showTicketDetails, filterAndRenderUpcomingTasks, loadEmpData, renderEmpTable, handleTaskStart

handleTaskEnd, handleReassignToUser, handleReassignToClient, handleClientResponse, openChatModal, loadExpenseHistory, handleExpenseSubmit, loadUsersData

setupFilters, addHeaderFilters, setupDateRangeFilter, renderActionButtons, loadPendingApprovals, cleanRemarksForDisplay, startNotificationPolling, stopNotificationPolling

loadReportForTab, handleReportDownload, handleUpdateSchedule, setFmsTeamTabsVisibility, activateFmsTab, parseFmsDateClean, isFmsFutureTask, escapeRegExp

matchesFmsUser, isMyFmsTask, isTeamFmsTask, loadFmsViewApp, populateFmsDropdowns, renderFmsTableApp, openFmsModalApp, clearFmsFilters

submitFmsTaskApp, closeFmsModalApp, renderAttachmentBadge, loadFormsPortal, populateFormsFilters, renderFormsTable, loadClientsPortal, renderClientsTable

checkFormsPortalPermissions, populateClientsDropdownFilters, showClientDetailsPopup, loadClientsPortal, renderClientsTable, renderFormsTable, isAppsScriptRuntime, performSubmission

toDateStr, readFile, processUpdate

Pasted code (3).js

Lines: 823; Detected functions/handlers: 30; UI IDs: 0

getSheetData, appendRowsToSheet, updateRowInSheet, saveBase64ImageToDrive, doGet, clientAuthenticate, checkPaymentRestriction, createBulkTicketsWithDetails

filterByDate, getClientTickets, getClientChecklists, getClientSocialTasks, getClientInvoices, getClientDashboardData, updateTicketStatusByClient, submitClientResponse

updateSocialPostStatusByClient, addClientRemarkToHistoryEntry, getTicketDetails, getChecklistDetails, getSocialTaskDetails, getSocialTaskHistory, getClientReportData, getMessagesForTask

postMessage, markTicketMessagesAsRead, checkForNewUpdates, markAsNotified, getLatestPendingDate, isTicketClosed

Pasted text (4).txt

Lines: 3,553; Detected functions/handlers: 53; UI IDs: 101

updateSidebarCounts, setIconTheme, toggleTheme, triggerCelebration, updatePrintDetails, showLoading, showSwalMessage, showRestrictionAlert

type, formatDateForDisplay, formatDateForInput, createFilterUI, setView, setupUI, fetchNextAvailableSlot, getFormattedDateString

handleLogin, handleLogout, updateNotificationUI, pollForUpdates, loadClientTickets, loadClientChecklists, triggerConfetti, fire

loadClientSocialTasks, loadClientInvoices, renderInvoicesTable, loadClientReports, renderReportCharts, renderAllTicketTables, getStatusBadgeClass, renderTicketTable

renderAwaitingResponseTable, getChatButton, renderChecklistTable, renderSocialTable, loadDashboardData, renderStatusDonutChart, renderActivityBarChart, renderPendingActions

renderRecentActivity, handleTicketAction, handleClientResponse, handleClientAddRemark, handleSocialPostApprove, handleViewTicketDetails, handleViewChecklistDetails, handleViewSocialDetails

openChatModal, addTicketRow, isEffectivelyClosed, filePromise, sendMessage

Pasted code (5).js

Lines: 514; Detected functions/handlers: 19; UI IDs: 0

getSpreadsheet, doGet, parseDateSafe, getFlexibleValue, getAdminReports, isTaskTrulyPending, createTicketInSheet, createTodoInSheet

getTicketSystemData, getScriptUrl, getSheetData, appendRowToSheet, toYMD, isDateInScope, isDateInLeaveRange, isValidTask

fmtDuration, mapTaskFlexible, toYMD

Pasted text (6).txt

Lines: 3,848; Detected functions/handlers: 43; UI IDs: 104

switchTab, getDaysCount, sortTasksByStatusPriority, refreshData, populateTableDropdowns, generateStatusFilters, updateStatusBtnLabel, toggleAllStatuses

populateExplorerDropdowns, isUserMatch, applyLocalFilters, renderNewAnalyticsCards, processData, renderOverviewMetrics, renderOverviewCharts, getDatesInRange

getDayName, isUserMatchRow, getClientFlexibleValue, loadUserExplorerProfile, calculateSubTableStats, loadClientExplorerProfile, renderTable, formatChildRow

loadDropdowns, openBatchModal, addTaskRow, submitBatch, parseDuration, fmtMins, renderClientEffort, renderInstantClosures

filterOverviewTables, exportEffortCSV, isClosedGroup, isInProgressGroup, isPendingGroup, fillSelect, filterTask, toYMD

filterByDateRange, getBreakdown, extractTimePart

31. Appendix B - Current Sheet Names Detected

File	Constant	Sheet/tab
Pasted text.txt	USERS	Users
Pasted text.txt	CLIENTS	Clients
Pasted text.txt	TICKETS	Tickets
Pasted text.txt	TICKET_HISTORY	Ticket_History
Pasted text.txt	ATTENDANCE	Attendance
Pasted text.txt	EXPENSES	Expenses
Pasted text.txt	LEAVE_REQUESTS	Leave Requests
Pasted text.txt	INTIMATIONS	Intimations
Pasted text.txt	EMP_MASTER	EMP Master
Pasted text.txt	MESSAGES	Messages
Pasted text.txt	FMS	FMS
Pasted text.txt	TODO	To-Do
Pasted text.txt	FORMS	Forms
Pasted code (3).js	CLIENTS	Clients
Pasted code (3).js	TICKETS	Tickets
Pasted code (3).js	CHECKLIST	CHECKLIST TASK
Pasted code (3).js	SOCIAL	Social Media
Pasted code (3).js	SOCIAL_HISTORY	Social_History
Pasted code (3).js	MESSAGES	Messages
Pasted code (3).js	USERS	Users
Pasted code (5).js	USERS	Users
Pasted code (5).js	CLIENTS	Clients
Pasted code (5).js	TICKETS	Tickets
Pasted code (5).js	FMS	FMS
Pasted code (5).js	TODO	To-Do
Pasted code (5).js	ATTENDANCE	Attendance
Pasted code (5).js	LEAVES	Leave Requests
Pasted code (5).js	INTIMATIONS	Intimations
Pasted code (5).js	INVOICES	Invoices
Pasted code (5).js	FOLLOWUPS	Followups

32. Appendix C - High Risk Technical Debt from Current MVP

Risk	Why it matters	Fix
Plain/default passwords	Account compromise risk	Hashed passwords and setup links
Spreadsheet as database	Slow, race conditions, weak constraints	PostgreSQL with indexes and transactions
Hardcoded business IDs	Developer must edit code for policy changes	Permission/settings tables
Repeated utility functions	Bug fixes inconsistent	Shared backend utility modules
Row-number IDs	Duplicate/conflict risk after deletes/sorts/imports	UUID + readable sequence codes
Public file URLs	Private documents may leak	Signed/permissioned storage
Free-text statuses	Wrong reports and filters	Status master + mapping
Large single HTML files	Hard to maintain and test	Component-based frontend
Direct dashboard scans	Slow as data grows	Reporting views/cache/materialization
Limited automated tests	Regression risk	Unit, integration and role-permission tests